

Examen final – Programación II

Se presenta un modelo orientado a objetos de un sistema para gestionar una academia de formación profesional.

A partir del siguiente diagrama UML, codificá todas las clases en Java utilizando anotaciones de Lombok (@Data, @Builder, @NoArgsConstructor, @AllArgsConstructor, etc.).

Relacionamientos

- Asociación entre Academia y Curso
 - Agregación entre Academia y Instructor
 - Composición entre Curso y Modulo
 - Herencia entre Persona y las clases Instructor y Estudiante
-

Requisitos adicionales

- Implementar la interfaz Registrable, que contiene el método registrar() e implementarla en la clase Inscripcion.
 - Utilizar las enumeraciones:
 - NivelCurso (BASICO, INTERMEDIO, AVANZADO) en Curso
 - EspecialidadInstructor (PROGRAMACION, DISEÑO, MARKETING, DATOS) en Instructor
 - Implementar la superclase Persona, con los atributos id, nombre y email, y heredarla en las clases Instructor y Estudiante.
-

Métodos obligatorios

- agregarCurso(Curso curso): agrega un curso a la lista de cursos de la academia.
 - buscarCursoPorNombre(String nombre): devuelve el curso cuyo nombre coincida.
 - asignarInstructor(Instructor instructor): agrega un instructor a la lista.
 - listarModulos(): imprime todos los módulos de un curso.
 - registrar(): muestra un mensaje de inscripción registrada con la fecha (implementado desde Registrable).
-

Instanciá con @Builder:

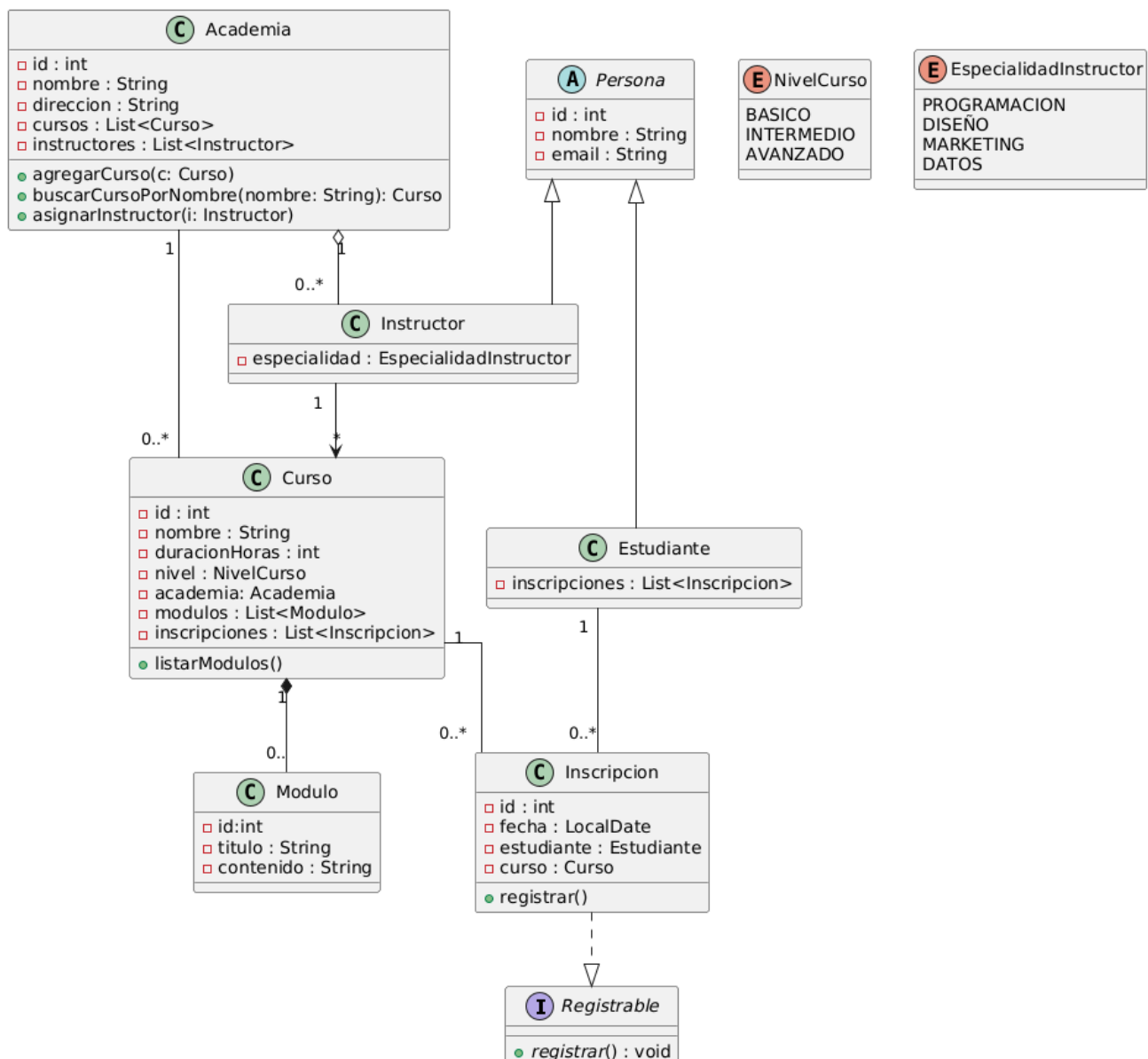
- 1 Academia
- 2 Instructores con especialidades distintas
- 3 Cursos con niveles distintos
- 1 Módulo por curso
- 1 Estudiante
- 2 Inscripciones

Persistencia con JDBC:

Se requiere persistir en base de datos la información de la clase Academia, sin incluir la lista de cursos ni la lista de instructores en esta etapa.

- Para ello, creá las tablas Academia e Instructor en la base de datos dbacademia, y generá la clase de conexión utilizando JDBC.
- Asegurate de definir correctamente la relación de clave foránea que vincule a los instructores con la academia correspondiente.
- Agregá los atributos necesarios en las clases Academia e Instructor para permitir su correcta persistencia (por ejemplo, el atributo academiald en la clase Instructor).
- Implementá la lógica para:
 - Guardar instancias de Academia e Instructor en la base de datos.
 - Realizar una consulta que recupere y muestre por pantalla la lista de instructores correspondientes a un ID de academia específico.

Diagrama UML



SQL para persistencia

```
CREATE DATABASE IF NOT EXISTS dbacademia;
```

```
USE dbacademia;
```

```
-- Tabla Academia
```

```
CREATE TABLE Academia (
```

```
    id INT PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    direccion VARCHAR(200) NOT NULL
```

```
);
```

```
-- Tabla Instructor (hereda de Persona, pero solo persistimos aquí lo necesario)
```

```
CREATE TABLE Instructor (
```

```
    id INT PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100),
```

```
    especialidad VARCHAR(50) NOT NULL,
```

```
    academia_id INT,
```

```
    FOREIGN KEY (academia_id) REFERENCES Academia(id)
```

```
);
```

Criterios de Evaluación

Criterio	Puntaje
Código compila sin errores	2 pts
Uso correcto de Lombok	2 pts
Relaciones bien implementadas	2 pts
Métodos implementados correctamente	2 pts
Instancias con Builder mostradas	2 pts
Persistencia con JDBC (guardar y consultar)	4 pts
Total	14 pts